

Structured Conditioning Mostly Does Not Help: A Pre-registered Account of the Exceptions

Richard Bao
richardbao419@gmail.com

Abstract

Conditioning a network means injecting a class, an action or a caption into its computation, and three mechanisms carry almost all of it: concatenation, feature-wise modulation (FiLM), and hypernetworks. Which to use is settled by sweeping, and the resulting tables disagree. We built the structured conditioner theory recommends, a group action whose composition is exact for any weights, and ran it through 25 pre-registered gates. It mostly does not help: fifteen gates returned negatives, both benchmarks whose task we did not design were losses, and the one prediction we made before running the experiment was wrong.

The exceptions are principled, and that is the contribution. Each mechanism reaches a different set of feature maps, and the polar decomposition says which: FiLM the stretch factor, a group action the rotation factor, a hypernetwork both. Each mechanism’s approximation error then has a closed form, fixed by which part the task needed and did not get. Two consequences are hard limits. No orthogonal conditioner can represent an eigenvalue of modulus $\neq 1$ in *any* representation, so conditioning that specifies content is out of reach for it at any parameter count. And exact composition forces linearity while content forbids it, so the two roles cannot share a channel; the minimal algebra carrying both is $\mathbb{C}^* \cong \mathbb{R}_{>0} \times U(1)$, which derives magnitude-times-phase modulation rather than proposing it.

Structure pays where the account says. The advantage is +0.6% on one condition and +46.6% on four ($p=9.1 \times 10^{-5}$), and absent at weak single conditions, so difficulty cannot explain it. Content conditioning fails by 8 $\times$, as the impossibility requires. The fitted half of the account breaks. Two short runs are meant to predict which tasks pay; both passed on a sixteen-control chain we built because the account favored it, and the advantage was flat at -1.1% . Structured conditioning pays when a task’s own conditions compose the way the operator does, which no dimensional criterion captures.

1 Introduction

Nearly every conditional model must inject something into its computation: a class, an action, a timestep, a text embedding. Call this conditioning. Diffusion models modulate every block on the timestep and class via adaLN [2]; visual reasoning, speech and RL systems modulate features on questions, speakers and goals [1]; world models condition latent dynamics on actions. Three mechanisms carry almost all of it: concatenation, feature-wise affine modulation (FiLM), and a hypernetwork that emits weights from the condition.

Which one to use is settled empirically. Papers train all three and print a table, and the tables disagree across domains. We know of no account that says which mechanism suits which task, so every new setting repeats the sweep, and a practitioner who reads two papers gets two answers. Geometric deep learning explains when symmetries of the *data* help, but conditioning is a different object: whatever structure exists lives in the condition space.

We set out to close that gap with a mechanism. Positional encoding supplies one: RoPE [4] and GRAPE [5] turn position into a rotation, and because position enters the rotation angle linearly, two positions compose exactly without ever being trained together. Carrying that from scalar positions to arbitrary conditions, in FiLM’s role and at FiLM’s cost, makes composition a property of the formula rather than something learned. It is the structured conditioner the theory recommends, and we expected it to win.

It mostly does not. Twenty-five pre-registered gates returned fifteen negatives. Both benchmarks whose task we did not design ourselves, a published PDE benchmark and a real analog compressor, were losses, to concatenation and to a hypernetwork respectively. Then we made a prediction before running the experiment instead of after. The setting was the high-dimensional condition space where coverage arguments say structure should matter most. The advantage came out flat at -1.1% . A practitioner reading only our positive results would reach for this operator and be wrong most of the time.

The exceptions are not noise, and characterizing them is what the paper contributes. Mechanisms differ less in how much they can express than in *what* they can express. A condition names one of two things, and the distinction runs throughout: it can name a *change* to apply, such as a pose offset, or the *content* to produce, such as a class or a caption. Those ask for different operations on the features. Any map splits into a rotation and a stretch, and each mechanism reaches essentially one part (Figure 1). Its error is whatever the task needed from the other one (§3).

Two consequences are hard limits, not tendencies. Rotations preserve lengths, so a conditioner built from them cannot shrink or grow any direction, and no change of coordinates rescues that because coordinates do not move eigenvalues. Producing content means suppressing features, so content is out of reach at any parameter count. Separately, exact composition forces the condition-to-modulation map to be linear, while content is not additive and forbids linearity. The two roles therefore cannot share a channel, and the smallest structure carrying both is a magnitude times a phase. The algebra hands you a conditioner, and it is the successor our negatives specify.

Where the account predicts a win, we measure one. The shape of the win confirms composition, where the margin alone could not. The advantage is 0.6% on one factor and rises to 25.5%, 38.7% and 46.6% on two, three and four, while every arm stays within 1.6% of the others at weak single conditions. Difficulty and strength cannot produce that (Figure 2). Content conditioning fails by 8 $\times$, as the impossibility requires. The polar type of a Navier-Stokes operator named which of our own arms would fail before we understood why. The fitted half breaks, and the -1.1% above is where. The condition it was missing turns out to be measurable in advance, by an instrument already sitting in our repository.

Contributions. (1) **A negative result, pre-registered.** Structured conditioning loses on most of the settings we tested: fifteen negatives across 25 gates with margins, tests and single-read splits committed before every run, including both benchmarks whose task we did not design. We report two verdicts that cleared their criteria as failures, because the criteria named the wrong baseline (§5, §6). (2) **What each mechanism can reach.** The reachable sets of FiLM, group actions, hypernetworks and complex modulation barely overlap, so parameter count is the wrong currency; the polar decomposition gives each one’s approximation error in closed form (§3). (3) **Two impossibilities that explain the negatives.** No orthogonal conditioner represents an eigenvalue of modulus $\neq 1$ in any representation, and no conditioner carries content and exact composition on one channel. The second derives magnitude-times-phase modulation as the minimal resolution and shows it is maximal (§3). (4) **The exceptions, established by shape rather than margin.** The advantage rises from 0.6% to 46.6% with the number of composed conditions and vanishes at one weak condition, which rules out the alternatives a fixed comparison cannot (§6). (5) **The account’s own falsification.** Its two screening quantities predicted a win on the one task where the prediction preceded the run, and the advantage was flat at -1.1% . We report the missing condition and the instrument that measures it (§7).

2 Related work

FiLM and conditional normalization. FiLM [1] and its normalized variants (conditional BN/LN, adaLN in DiT [2]) apply per-feature affine transforms generated from a condition. All are diagonal. We keep FiLM’s role and enrich only the operator.

Structured operators in parameter-efficient fine-tuning (PEFT). Low-rank, orthogonal, and butterfly operator families are well studied as weight fine-tuning: LoRA [6], OFT [7], BOFT [8], HRA [9], Group-and-Shuffle [10]; He et al. [11] give a unified view. These learn one fixed transform per task. Our setting differs on every axis that matters: per-sample, input-conditioned, activation-space, and evaluated for compositional generalization. Hypernetworks [3] generate weights from conditions and inherit the memorization behavior we quantify.

Position encodings as group actions. RoPE [4] rotates query-key pairs by angles linear in position; GRAPE [5] generalizes this to group representations with exact relative laws. The mechanism is theirs; our contribution is the transplant to arbitrary condition vectors in FiLM’s role, with a learned basis, and the budget-fair compositional evidence.

Compositional generation. Montero et al. [12] showed reconstruction-trained generative models fail on unseen factor combinations. Score and energy composition [23] composes concepts across separate conditional passes; we compose conditions inside the operator, in one pass. Symmetry-based representation learning [13, 14] argues latents should carry group actions; we supply the conditioning-side mechanism.

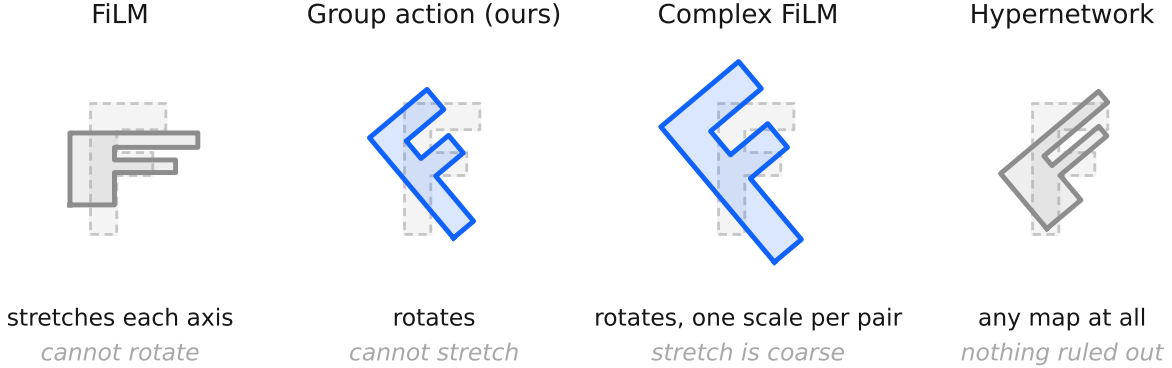


Figure 1: What each conditioner can do to your features. Conditioning applies a map to the activations, and any map can be split into a rotation and a stretch. The dashed shape is the input; the solid shape is what comes out. FiLM stretches the axes and cannot turn anything; a group action turns things and cannot stretch them; complex modulation does both but with one scale for each pair of features; a hypernetwork does anything. These are different abilities, not different amounts of the same ability, which is why parameter count does not predict which one works.

Nearest prior work. We searched deliberately for work that would invalidate this paper. The closest hits: Feature-space rotations with exact composition exist for geometric transformations [17]; abelian exponential-map operators for disentanglement [22]; attribute operators for recognition [26]; multilinear attribute mixing for unseen combinations [27]; per-sample amplitude-and-phase activation modulation in periodic implicit networks [20]; and composition by complex phase addition in knowledge-graph embeddings [21]. Rotation-structured latent world models with latent-prediction losses originate with Quessard et al. [15], Caselles-Dupré et al. [16], and the homomorphism autoencoder [18], whose composition property is emergent rather than imposed; latent consistency objectives are standard in model-based RL [19]. Distillation moves guidance into the conditioning path [25], and prior work documents CFG’s high-scale distortion [24].

We claim none of these components, and several of them are instances of what we analyse: the amplitude-and-phase modulation of Chan et al. [20] and the phase composition of Sun et al. [21] are the split our §3 derives, arrived at by construction rather than by necessity. What we could not find anywhere is an account of *which* mechanism suits which task. The operator families above are studied as parametrizations and compared by benchmark; none of the work we surveyed states what a mechanism can and cannot represent, or predicts its own failures. We searched deliberately for such an account and report its absence as a negative search, not as a proof that none exists. That account, its two impossibilities, and the pre-registered programme that tests and partly falsifies it are what we contribute.

3 Which maps a mechanism can reach

Mechanism choice looks like a matter of taste because the usual currency, parameter count, does not measure the thing that decides it. This section replaces that currency.

Conditioning multiplies the features by a matrix that depends on the condition, so asking what a mechanism can do means asking which matrices it can produce. Call that its *repertoire*. FiLM’s repertoire is the diagonal matrices, a hypernetwork’s is all of them, and the two operators we build in §4 give the rotations (**CGA**) and rotations with one scale per feature pair (**Complex FiLM**).

Any matrix splits into a rotation followed by a stretch, and that split is unique. This is the polar decomposition, and it is the right way to compare repertoires because each mechanism reaches essentially one of the two parts (Figure 1). Test error then breaks into three pieces: how far the truth sits from the repertoire, how much data it takes to find the right element of it, and how badly the model does at conditions it never trained on. Write these $E_{\text{approx}} + E_{\text{est}} + E_{\text{extrap}}$. Two facts follow, and one of them rules out a class of designs outright. Full derivations are in the released `THEORY.md`; §6 measures every quantity previewed here.

Approximation error is about which maps you can make, not how many you can make. The repertoires barely overlap. A matrix that is both diagonal and a rotation has to have entries ± 1 , so FiLM and a group action share only 2^d matrices out of two families of the same size. Switching between them does not shrink the repertoire, it *moves* it, which is why parameter counts mislead here: Complex FiLM uses $0.85\times$ FiLM’s parameters and reads as a 15% trim when it is a substitution of one ability for another. Write the required map as a rotation times a stretch, $T = US$. Each mechanism’s error then comes out exactly, not as a bound: $E_{\text{approx}} = 0$ for a hypernetwork, $\|T^* - \text{diag } T^*\|_F$ for FiLM, $\|S^* - I\|_F$ for a group action, and a per-pair residual for Complex FiLM. The middle one is the classical closest-rotation problem, solved by Schönemann in 1966. Read it plainly: a group action’s error *is* however much stretching the task needed and did not get, which is why a rotation-only conditioner fails at content by $8\times$ at any parameter count (§6).

One impossibility, and its converse. Rotations preserve lengths. So if the map a task needs shrinks or grows any direction, meaning it has an eigenvalue whose size is not 1, no group action can produce it. Changing coordinates does not rescue this, because a change of coordinates leaves the eigenvalues alone. Content conditioning needs $|\lambda| \rightarrow 0$. Its $8\times$ failure is structural. The converse matters as much. Approximation error refers to the transformation required *in the coordinates the encoder produced*, so it is a property of the task and representation together, not of the task alone. A learned representation can repair a rotation wearing the wrong basis and can never repair a change of scale. This predicts that freezing the representation is fatal, which §6 confirms, and it carries a design rule: the operator must be the *only* conditioning path, since a parallel unstructured route removes the pressure to find such coordinates. Our own hybrid arm, which contained a concatenation path, scored +1.7% against concatenation alone, a tie.

Exact composition bounds extrapolation, and only on the compact factor. When the operator is built from rotations, small errors in the learned weights cause only small errors in the operator, never amplified: $\|\hat{T}(c) - T(c)\|_F \leq \|A\| \|\hat{W} - W\| \|c\|$. The bound depends on $\|c\|$ alone and not on how far c sits from the training conditions: reaching an unvisited *combination* costs nothing beyond reaching the same magnitude, and if the training conditions span the condition space then W is identifiable and the error vanishes with data. Add a stretching direction and that protection is gone: error grows like e^s , which is why the magnitude channel needs a clamp while the phase channel composes to 10^{-6} , and why guidance degrades linearly in strength on the rotation channel and exponentially on the magnitude one.

Complex FiLM is forced, not chosen. The two conditioning roles cannot share a channel, and the algebra says what to do instead. Exact composition requires $A(c_1 + c_2) = A(c_1) + A(c_2)$ in the exponent, and a continuous map that turns sums into sums is linear. That is Cauchy’s functional equation, and it says no exactly-compositional conditioner can have a nonlinear head. Content runs the other way. Specifying what to produce is not additive, since composing “cat” with “dog” is not “cat + dog”, so the condition-to-modulation map has to be expressive; the fully linear variant scores 0.19 in the content role against the expressive variant’s 0.97 (§6). The two requirements cannot hold on one channel, so the representation has to split. This is forced, not preferred. The smallest algebra carrying both an expressive and an exactly-compositional direction is $\mathbb{C} = \mathbb{R} \oplus i\mathbb{R}$, whose multiplicative group factors as $\mathbb{C}^* \cong \mathbb{R}_{>0} \times U(1)$: magnitude and phase. Compactness fixes which carries which. Rotations are bounded, so exactness survives on them; e^s is unbounded, needs a clamp, and a clamp is nonlinear. We measure composition error of 10^{-6} while $|s| \leq 4$ and order 1 the moment the clamp engages. It is also as large as this can get. Exact composition needs the generators to commute, the largest commuting family of $d \times d$ matrices has d dimensions, and magnitude-plus-phase over $d/2$ pairs already uses all d . Anything bigger gives up commuting, and with it exactness at arbitrary weights.

4 Two conditioners the account names

The account of §3 names two operators: one that reaches the rotation factor exactly, and one that reaches both factors at the cost of exactness on half of them. Both already exist in special cases, and writing them in this form is what makes them testable.

Let $x \in \mathbb{R}^d$ be features and $c \in \mathbb{R}^k$ a condition. FiLM computes $y = \gamma(c) \odot x + \beta(c)$; hypernetworks emit $W(c)$. All are instances of

$$y = T(c)x + \beta(c), \quad (1)$$

and they differ only in where the per-sample operator $T(c)$ comes from.

Call T *compositional* when applying two conditions at once equals applying them in sequence: $T(c_1 + c_2) = T(c_1)T(c_2)$, with $T(0) = I$. Getting that exactly, and not just approximately after training, takes one construction.

Proposition 1 (Exact composition by construction). *Build the operator as $T(c) = P \exp(A(Wc))P^\top$, where W maps the condition linearly into a set of matrices that all commute with one another, A places it there, $\exp$ is the matrix exponential, and P is a fixed rotation. Then T is compositional for every choice of W and P . If the commuting matrices are antisymmetric, every $T(c)$ is a rotation.*

Proof. $\exp(X)\exp(Y) = \exp(X+Y)$ whenever X and Y commute, and the condition enters $\exp$ through a linear map, so adding conditions adds exponents. Sandwiching by P preserves products, and $\exp(0) = I$. $\square$

The point is where composition comes from. Usually a network learns it from data that shows conditions together. Here it is true of the formula, so it holds at initialization, during training, and at conditions far outside the training set. Training decides only which directions the condition turns and how fast.

One caveat is easy to miss. Composition pins down T at an unseen condition only if that condition is a sum of ones you trained on. Train on conditions that cannot be added up to reach the test ones and exactness buys nothing, which is why a very large compositional gap can mean the test set is out of reach rather than that there is an opportunity.

Special cases. Different choices recover mechanisms already in use:

commuting family	condition	P	recovered mechanism
$\text{diag}(\mathbb{R}^d)$	c	I	<i>exponential FiLM</i> : $y = e^{Wc} \odot x$ (compositional)
planar rotations	$c = n$ (scalar position)	I	RoPE [4]
planar rotations	$c = n$, learned planes	learned	multiplicative GRAPE [5]
planar rotations	general $c \in \mathbb{R}^k$	structured	this work (FiLM’s role)
scales $\times$ rotations	general c	canonical	Complex FiLM (§6)

Standard FiLM reaches the same diagonal operators through an MLP head $\gamma(c)$. The operators match; only the composition law is missing, and §6 shows that its absence is what breaks FiLM on unseen combinations.

Cost. For planar rotations the exponential is a sine and a cosine per pair, so no matrix exponential is ever computed: $3d$ FLOPs, no matrix exponential. The basis P is the whole budget question. A dense learned P costs $4d^2$ FLOPs per sample, which is $1.52\times$ FiLM’s entire conditioning path at $d=128$. Two fixes work. A structured orthogonal P (five block-diagonal Cayley layers with fixed shuffles, following Group-and-Shuffle) brings the total to $1.11\times$ FiLM, with 4,800 parameters against the 8,128 dimensions of $\text{SO}(128)$; that is enough, because P only needs the planes the condition acts on. Inside a transformer no explicit P is needed, since the adjacent dense projections absorb it exactly as RoPE relies on the query and key projections rather than on a basis of its own. Linearity also gives a strength dial for free: $T(c)^w = T(wc)$, so raising the operator to a power is the same as scaling the condition, which §6 uses in place of CFG’s extrapolation between a conditional and an unconditional prediction. Unit tests pin identity at initialization and at $c=0$, orthogonality, and exact composition. Complex FiLM extends the algebra to $\text{diag} \times$ rotations: each feature pair is multiplied by $m e^{i\theta}$, with magnitude from FiLM’s expressive head (the content channel) and phase linear in the condition (the composition channel), at $0.84\times$ FiLM’s cost in our harnesses.

5 Pre-registered, budget-fair protocol

Vocabulary. An *arm* is one conditioning method, trained under the shared recipe. A *suite* is one experiment: a fixed backbone, dataset, and training loop that all of its arms share. A *gate* is that suite’s pass-or-fail rule, written down before the run as a conjunction of *criteria*: typically a margin over the strongest baseline on unseen combinations, a no-regression criterion on the training distribution, and a budget criterion. A suite passes only if all of its criteria pass, and we report the outcome either way.

Design. We commit every suite’s success margin, statistical test ($N=10$ seeds, one-sided Mann–Whitney $p \leq 0.01$, Cliff’s $|\delta| \geq 0.474$) and splits to version control before its decision run. Each test split is read once, after all selection on a validation split. All arms share one backbone and one training recipe. A shared counter enforces the budget: our conditioning path stays within $1.20\times$ FiLM’s FLOPs and $1.05\times$ the smallest hypernetwork baseline’s parameters, while baselines reach $48\times$ our parameter count. We record deviations as dated amendments in the committed specifications, plus one accounting erratum that our own audit caught and that downgraded a favorable verdict.

Table 1: Error on never-trained condition combinations: mean (seed sd) over 10 seeds. CGA is lowest in every column, with complete seed separation against the strongest hypernetwork baseline ($p=9.1\times 10^{-5}$, $\delta=-1.0$). OOD means out-of-distribution: a condition combination never seen in training. Every column except 3D Shapes passed its full pre-registered gate; 3D Shapes failed on a separate training-fit criterion (text). The last column reruns the dSprites setup with Complex FiLM. **Columns are not comparable to one another:** the two synthetic suites report operator MSE, the rest report per-pixel MSE.

Conditioner	Aligned	Hidden basis	dSprites	+shape	3D Shapes	Complex FiLM
FiLM	$1.3 (0.1)\times 10^{-2}$	$1.6 (0.04)\times 10^{-2}$	$3.0 (0.2)\times 10^{-3}$	$5.9 (0.2)\times 10^{-3}$	$4.5 (1.0)\times 10^{-3}$	$3.1 (0.2)\times 10^{-3}$
Concat-MLP	$1.5 (0.3)\times 10^{-2}$	$1.1 (0.4)\times 10^{-2}$	$2.9 (0.1)\times 10^{-3}$	$6.3 (0.2)\times 10^{-3}$	$5.4 (5.2)\times 10^{-3}$	–
Cond-LN	$2.4 (0.1)\times 10^{-2}$	$2.7 (0.05)\times 10^{-2}$	$3.2 (0.2)\times 10^{-3}$	$6.1 (0.3)\times 10^{-3}$	$8.6 (13.1)\times 10^{-3}$	–
Hypernetwork	$1.9 (0.3)\times 10^{-3}$	$3.4 (0.9)\times 10^{-3}$	$3.8 (0.3)\times 10^{-3}$	$7.0 (0.4)\times 10^{-3}$	$9.3 (2.3)\times 10^{-3}$	$3.8 (0.2)\times 10^{-3}$
Dyn. low-rank	$5.7 (0.5)\times 10^{-3}$	$6.1 (0.4)\times 10^{-3}$	$5.1 (0.9)\times 10^{-3}$	$8.1 (0.4)\times 10^{-3}$	$2.0 (0.4)\times 10^{-2}$	–
MLP-head (abl.)	–	$1.1 (0.2)\times 10^{-3}$	$2.7 (0.2)\times 10^{-3}$	$5.0 (0.2)\times 10^{-3}$	$3.3 (0.8)\times 10^{-3}$	–
Complex FiLM (linear mag.)	–	–	–	–	–	$2.0 (0.1)\times 10^{-3}$
Complex FiLM	–	–	–	–	–	$2.0 (0.2)\times 10^{-3}$
CGA (ours)	$7.4 (2.0)\times 10^{-4}$	$3.3 (2.2)\times 10^{-8}$	$1.8 (0.1)\times 10^{-3}$	$4.0 (0.1)\times 10^{-3}$	$1.2 (0.2)\times 10^{-3}$	$1.8 (0.1)\times 10^{-3}$

Threats to validity, and the design answer. The benchmarks are self-constructed, so we built them to remove judgment from scoring: dSprites and 3D Shapes are deterministic factor-to-image maps, so every held-out condition has a unique ground-truth image and pixel error is an objective compositional metric, with no perceptual score and no human rating. Pre-registration removes outcome-contingent analysis, and fifteen of the twenty-five verdicts below are negatives, one of which falsified our own headline prediction. Every number here regenerates from the committed logs with one command per suite.

Arms. FiLM; conditional LayerNorm; Concat-MLP; a dense hypernetwork $I+\Delta W(c)$; a dynamic low-rank map $I+A(c)B(c)^\top$; the Lie operator; Complex FiLM (both variants); and an MLP-head ablation of our own method (same basis, same operator family, more compute, no linearity).

The experiments. Two synthetic suites. Learn $y = M(c)x$, where c picks a combination of 8 rotation primitives. The second hides the primitives behind a random basis, so the operator has to find them, and additionally tests all 56 never-trained triples. **Three image suites** on dSprites and 3D Shapes. Encode a real image, apply $T(\Delta)$ to the learned latent for a factor change Δ , decode, and score pixel error against the true transformed image; OOD means never-trained *types* of two-factor change. Two of the three add a categorical shape factor. **Content.** A small diffusion transformer generates dSprites images from a full factor specification, so the condition names what to draw rather than how to change it. **Rollouts.** A world model applies the conditioning arm recurrently, one action per step, tested at horizons 10 and 20 after training on 3. **The successor.** Complex FiLM, through the dSprites and content setups. **Guidance.** Condition-dropout training, then classifier-free guidance at strength w for FiLM against condition powering for Complex FiLM, scored against deterministic ground truth at each strength.

6 Evidence: the exceptions, and the rest

Table 1 reports every suite, and the negatives outnumber the wins. Each result below tests something §3 asserts: (1) to (3) the composition claim and its mechanism, (4) the derived conditioner, (5) and (6) the impossibility and the fit penalty it implies, and (7) to (9) the representation, rollout and guidance consequences. Four are settings where structure buys nothing, and they are as informative as the wins because the account names them in advance. Every verdict comes from a gate committed before its run.

(1) Composition is exact, and the weights say what each condition does. On the hidden-basis suite, CGA reaches 3.3×10^{-8} error on unseen pairs and 5.3×10^{-8} on never-trained triples. The best baseline sits five orders of magnitude higher, and every baseline degrades as combinations lengthen (Figure 4). Learned angles match the true generators to 2×10^{-4} rad, so reading the weights tells you which plane each condition rotates and how fast (Figure 5).

Every conditioner degrades on unseen condition combinations. CGA degrades least.

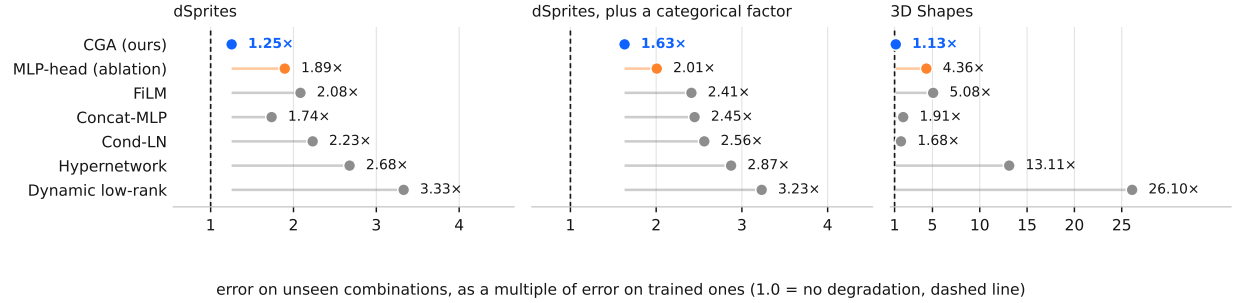


Figure 2: How much worse each conditioner gets when conditions are combined in ways it never saw, as a multiple of its own error on trained combinations. A value of 1.0, the dashed line, would mean no degradation at all. CGA sits closest to it on all three image suites, and the two conditioners that can express any linear map sit furthest.

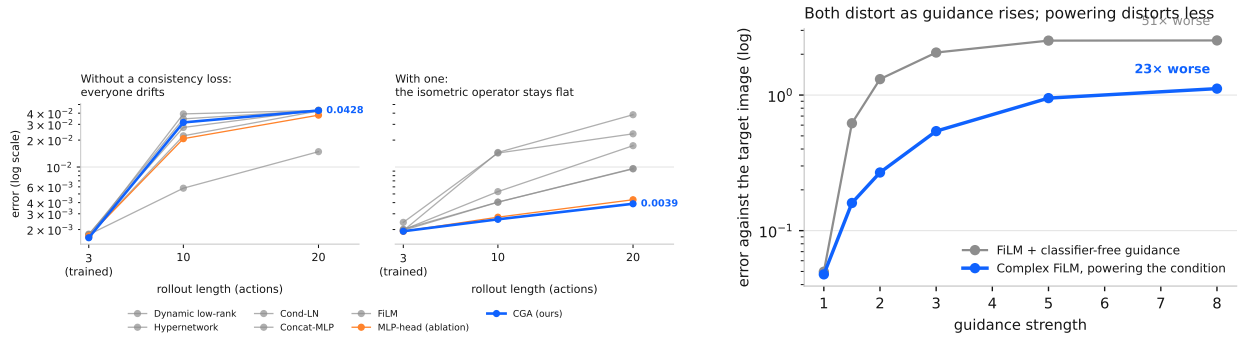


Figure 3: **Left:** error against rollout length, trained only to length 3. Without a consistency loss every conditioner drifts at much the same rate. Adding one separates them: the isometric operator flattens out while the baselines keep climbing. **Right:** what turning guidance up does to the generated image. Both distort, since the target here is deterministic and there is no distribution to sharpen, but classifier-free guidance distorts twice as fast.

(2) The advantage comes from composing conditions, and its shape shows it. Comparing arms on a fixed evaluation cannot separate a compositional mechanism from an arm that is simply better. We therefore swept the number of simultaneously changed factors and their magnitude independently, and registered in advance that the advantage must vanish for a single weak condition and grow with the number composed. At the trained magnitude the operator gains 0.6% over the best additive arm on one factor, then 25.5%, 38.7% and 46.6% on two, three and four ($p=9.1 \times 10^{-5}$, $\delta=-1.00$); at weak single conditions every arm falls within 1.6% of the others. An advantage that is absent at one condition and rises with the number composed cannot be difficulty, strength, or a generally stronger arm.

The magnitudes follow. On dSprites every arm fits the training distribution equally well, which isolates composition: CGA cuts error on unseen combinations by 53.8% against the strongest baseline at 39x fewer conditioning FLOPs, and by 42.9% once a categorical shape factor joins the condition. Moving from trained to unseen combinations costs CGA 1.25x, or 1.63x with the shape factor, against 2.1–3.3x for every baseline. We call that multiplier the *compositional gap*.

(3) The linear map is the mechanism, and the operation class rather than the parameter count explains the rest. The ablation keeps the basis and the operator family, spends more compute, and swaps only the linear map for an MLP. It loses by $4.3 \times 10^4 \times$ on synthetic triples and 1.55x on dSprites. The two hypernetwork arms can express any operator and post the largest compositional gaps of any arm on every image suite, 2.7–3.3x on dSprites and 13–26x on 3D Shapes against CGA’s 1.13–1.63x.

We first read this as capacity memorizing the training combinations, and our own audit does not support that reading.

Sorting the arms by parameter count does not sort them by error: the 2.1M hypernetwork beats the 298K low-rank arm on dSprites (0.00383 against 0.00508) and on 3D Shapes (0.00928 against 0.01978), and the 83K Concat-MLP often beats the 50K FiLM. What survives is weaker. The unstructured arms are usually worst on held-out combinations without being ordered among themselves, which points at which maps an arm can express and away from how many parameters express them (§3).

(4) Complex FiLM covers both conditioning roles at lower cost. Complex FiLM multiplies feature pairs by $m e^{i\theta}$, giving content to the magnitude channel and composition to the phase. Under two pre-registered gates it beats FiLM by 36% on unseen change types at fit parity and $0.84\times$ FiLM’s cost, and matches FiLM on content generation at $0.97\times$ its error (pre-registered non-inferiority margin $1.10\times$), where the pure rotation failed by $7\times$. Giving the magnitude channel a linear map instead of an MLP fails that same content metric (0.19 against FiLM’s 0.04), so content needs the expressive head. FiLM is the magnitude channel on its own; the full complex multiplication does both jobs.

A third gate confirms it outside the disentanglement benchmarks. On a 2D Navier-Stokes surrogate conditioned on four physical parameters (viscosity, drag, forcing and background advection, with the initial vorticity field supplying all of the content), Complex FiLM passes every criterion on fresh seeds: 45.4% below FiLM, 64.6% below the best unstructured arm at $\delta=-1.00$, and an in-distribution fit ratio of **0.938** $\times$, the first arm here to fit better than the unstructured baseline, replicated across independent seed sets. The claim covers Complex FiLM and not the pure group operator: its magnitude head is nonlinear in c , so composition stays exact on the phase channel only. It buys fit by spending half the guarantee, and the arm that keeps the guarantee on both channels failed this criterion twice.

(5) Content is provably out of reach for an orthogonal conditioner. In the content suite, rotation conditioning fails: pixel MSE 0.31 against FiLM’s 0.04, an $8\times$ regression, and the ablation fails identically. A proof explains it. Producing content means suppressing features, so the required transformation carries eigenvalues of modulus far from 1; every rotation leaves every eigenvalue at size 1; and changing the representation leaves the eigenvalues alone. No orthogonal conditioner can carry “generate this image” in *any* representation, and co-training cannot repair it (§3).

(6) The remaining fit penalty tracks content demand. Where the condition supplies some content, the operator fits worse in distribution than an unstructured arm. On 3D Shapes CGA posts the lowest error on unseen combinations (87.5% below the hypernetwork) and still trips the pre-registered no-regression criterion at $1.46\times$, and on the physics task the same criterion fails at $1.118\times$. Two registered relaxations that keep exact composition, a scaling generator and a learned diagonal conjugating the orthogonal basis, do not fix it: the conjugated operator looked like a cure on validation ($0.962\times$) and reached $1.1007\times$ on the held-out split, missing by 0.065 percentage points.

A magnitude channel does, but only where the condition supplies no content. Complex FiLM reaches $0.938\times$ on the physics task, whose four parameters act on content the input already carries; on 3D Shapes, whose condition includes a categorical shape swap, it reaches $1.542\times$, worse than the pure rotation, while still passing the compositional criteria by 84.5%. The penalty tracks how much content the conditioning must supply. This family suits conditions that *transform* content supplied elsewhere, and no variant we tested rescues it otherwise.

(7) The advantage requires a co-trained representation. We ran the same dataset, held-out pairs, arms and budget counter twice, varying whether the representation trains alongside the operator. This is not the single-variable comparison we once called it: the co-trained run optimizes and scores in pixel space, the frozen run in latent space, so objective and metric differ too, and only the within-run ratios are defensible. Co-trained, CGA sits 87.5% below the hypernetwork on unseen combinations. Frozen, against a plain autoencoder shared unchanged by every arm, it lands 60.1% *worse* while using $48\times$ fewer conditioning parameters, and fits $2.43\times$ worse in distribution against a $1.10\times$ ceiling. Complex FiLM fails there too, so the rotation channel is not the culprit.

The advantage is therefore a property of the operator and its representation together, and bolting it onto a frozen general-purpose encoder, the pattern latent-space editing deploys, loses. CGA still posts the smallest compositional gap of any arm that fits, $1.39\times$ against the hypernetwork’s $2.11\times$, so it degrades most gracefully and still loses: grace does not recover a fit handicap.

(8) Rollouts need a consistency loss, and the operator alone does nothing. In the rollout suite every arm compounds error at the same rate, because each step adds a small mismatch between the predicted and the true latent. A contracting operator shrinks past mismatches; an a rotation preserves them, so they accumulate. Training with

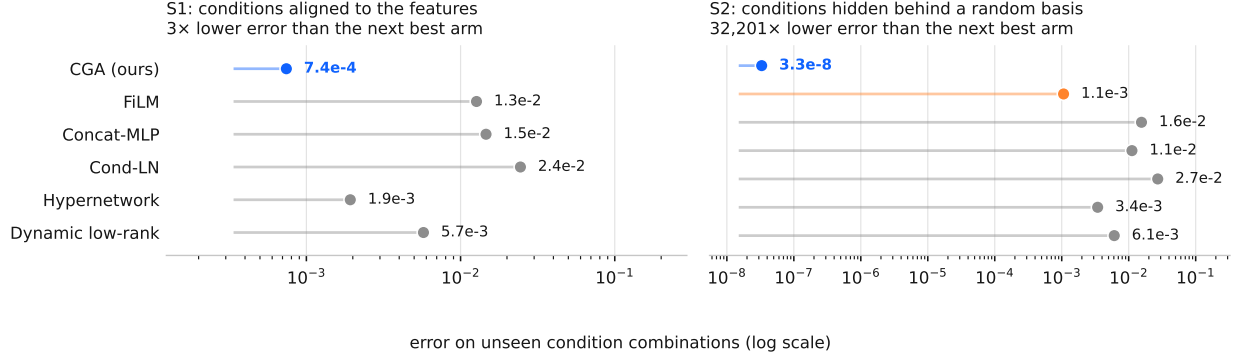


Figure 4: On the synthetic suites, where the conditions form a group exactly, the gap is not a matter of percentages. Each dot is one conditioner’s error on unseen combinations; note the log axis. Hiding the conditions behind a random basis (right) costs CGA nothing, because it learns the basis, while the gap to the next best arm grows to five orders of magnitude.

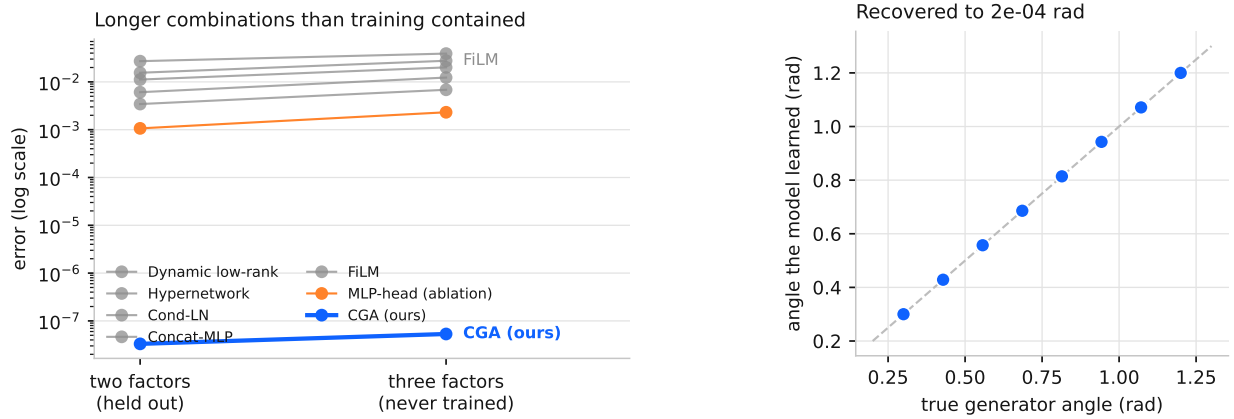


Figure 5: **Left:** what happens when a test combination is longer than anything in training. Every baseline gets worse. CGA does not move, because composition is built into its parametrization and was never learned. **Right:** the angles the model learned for each condition, against the true generators of the data. Reading the weights tells you which plane a condition rotates, and how fast.

$\lambda \|T(a)z_s - z_{s+1}\|^2$, applied to every arm equally, inverts the picture: the rotation operator’s horizon-20 error goes flat at 0.0039 (growth 2.0×) against the hypernetwork’s 0.038 (growth 19.8×) and FiLM’s 0.017 (Figure 3, left). The loss is standard in model-based RL [19], and rotation transitions with latent-prediction objectives appear in the homomorphism autoencoder [18]; giving every arm the same loss at the same budget shows that neither ingredient fixes rollouts alone. A null result sharpens the mechanism: shrinking the latent by a fixed factor each step, swept from 0.003 to 0.1, leaves horizon-20 error unchanged, so whatever removes the accumulated mismatch must depend on the state. A separate single-step criterion fails, so this suite counts as a negative in Table 1 despite the horizon-20 result.

(9) Guidance costs one forward pass, and holds only on a deterministic target. CFG’s distortion at high scales is documented [24]; our benchmark isolates it against an exact alternative. Turning the condition up through the phase channel grows error 23.7× from strength 1 to 8 against CFG’s 50.7× ($p=1.6 \times 10^{-4}$), stays 2–5× lower at every strength above 1, and needs no unconditional pass and no distillation [25] (Figure 3, right). Neither method improves over strength 1 here, because the target is deterministic and no distribution exists to sharpen, exactly as the suite’s registered caveat stated.

The sweep in (2) refutes the other half of what we registered. We had argued that additive aggregation, which is

Table 2: The two screening quantities against the outcome, for every task we ran. The fit ratio is the tested arm’s in-distribution error over the best-fitting baseline’s; the gap is that baseline’s held-out error over its own in-distribution error. Both are read off the baselines, since an arm cannot be its own yardstick. Predictions use $\text{fit} \leq 1.20$ and $\text{gap} \geq 1.5$. The last row is the only one where the prediction preceded the run, and it is the only miss.

Task	Fit ratio	Gap	Predicted	Outcome
dSprites	0.986×	2.68×	win	won 53.8%
dSprites + shape	1.019×	2.56×	win	won 42.9%
3D Shapes	1.457×	13.11×	loss	lost on fit
Navier-Stokes, rotation	1.337×	6.74×	loss	lost on fit
Navier-Stokes, Complex FiLM	1.122×	6.74×	win	won 64.6%
Latent editing, frozen	2.433×	2.11×	loss	lost 60.1%
PDEBench reaction-diffusion	2.381×	2815×	loss	lost to Concat-MLP
Audio effects, 4 controls	1.000×	1.13×	no gain	tied
Audio effects, 16 controls	1.009×	1.84×	win	flat at −1.1%

what a value-sum over condition tokens performs, should degrade as $\frac{1}{2} \|\sum_i A_i\|^2$ and therefore worsen with conditioning *strength*. The advantage instead reverses beyond the trained scale, reaching −14.8%. Rotations are periodic, so past the trained angle the operator wraps and the error oscillates. The bound of §3 survives as an upper bound linear in $\|c\|$, but it is loose exactly where we relied on it. Exact composition buys accuracy when stacking conditions at the scale trained on, and gives no strength dial.

7 Turning the account into a screen, and watching it break

Two short runs estimate both terms. Choose the mechanism whose reachable factor matches the polar type the task requires; then adopt exact composition only if the evaluation visits unvisited conditions. Short runs estimate both quantities: in-distribution fit ratio for the first, compositional gap for the second. Table 2 applies both to every task we ran. They separate the wins from the losses, and each is independently observed to fail while the other passes: the audio chain had a fit ratio of 1.000 and no gap, PDEBench an enormous gap and an unusable fit. A one-factor account cannot produce that pattern.

The threshold is softer than the separation. Both quantities need a yardstick arm, and the two defensible choices disagree: measured against the baseline each gate nominated in advance, the Navier-Stokes Complex FiLM run fits at 0.938×, and measured against whichever baseline fits best, at 1.122×. The wins and losses stay separated under either, but the dividing line moves from 1.05 to roughly 1.20. We report the stricter yardstick throughout Table 2 and treat the threshold as fitted rather than derived.

The account also covers results the capacity framing could not. The synthetic Navier-Stokes operator is diagonal in Fourier, so we can compute its polar factors instead of estimating them: viscosity and drag are pure stretch (phase 0.000), background advection is pure rotation (log-magnitude 0.000, phase 1.594). A task needing both factors is one a pure rotation cannot fit, and that is what happened, to our own arm. The rotation failed on fit while Complex FiLM, which reaches both, fit at 0.938× and won by 64.6%. The polar type named which of our arms would fail before we knew why.

The two checks are not sufficient, and the first out-of-sample test says so. We calibrated both thresholds on the comparisons they summarise, so that agreement is consistency, not predictive accuracy. We then built the case the account favors: a sixteen-control parametric chain, the high-dimensional regime where coverage arguments say structure should matter most. Both checks passed before the run, with a compositional gap of 1.83× and a fit ratio of 1.011×, so the account predicted a win. The measured advantage was flat at −1.1% across one, two, four and eight simultaneous controls.

A third condition is missing, and the instrument for it was already in the repository. A rigging check we had added only to confirm a task was not built to flatter the operator found 0 of 12 control pairs matching the operator’s form. That is the previous paragraph’s class test, pointed at the *task* instead of the mechanism. It called the outcome before training. The likeliest reason, though we have tested it only on this one failure, is that dSprites factors are independent coordinates of the generative process, so combining two changes yields a well-defined image and composition holds

exactly in factor space, whereas the chain’s saturation and dynamics stages interact, so the combined effect is not the composition of the individual effects. There is then nothing for exact composition to be exact about, however large the condition space or the gap. Structured conditioning pays when the task’s own conditions compose the way the operator does, which is narrower than any dimensional criterion and is measurable in advance.

What to do with it. Two short runs decide the question. Fit one high-capacity conditioner, read its effective operator, and let the closed forms above give each mechanism’s approximation error; measure the compositional gap on your own evaluation; then check that the task’s own conditions compose the way the operator does. Reach for a group-structured conditioner when all three hold: the condition names a change to apply rather than content to produce, held-out combinations are meaningfully harder than trained ones, and the representation trains alongside the operator. Keep FiLM otherwise. Complex FiLM is the drop-in where the condition transforms content the input already supplies, at lower cost than FiLM. In rollouts, pair the operator with a latent-consistency loss, since neither works alone.

Honest status. The mathematics is elementary: Procrustes, Cauchy, and a standard bound on how much the matrix exponential can amplify an error. We calibrated the thresholds quoted above on the same nine tasks they summarise, so the framework is consistent with our evidence rather than validated out of sample, and the tasks themselves were not pre-registered even though the splits were. We state the rule so that it can be falsified, and give in §8 the experiment that would do it.

8 Limitations

All evidence comes from controlled 64×64 benchmarks with known factors, width-128 models and a single conditioning site, so we validate nothing at production scale; the natural deployment experiment is the adaLN site of a text-conditioned diffusion transformer. Learned latents do not support exact group action, so the advantage on real images is the 1.8–8× error reduction quoted above rather than the $10^5\times$ of the synthetic suites, and 3D Shapes shows orthogonality can cost in-distribution fit on rich scenes. Conditions far from any group action, such as open-ended text, remain untested.

The guidance result does not transfer to a multimodal target. Our benchmark measures distortion against a deterministic target, so it cannot see the distribution-sharpening CFG exists to provide. On a second harness where $p(x | c)$ is uniform over 64 cells, an unregistered single-seed pilot with no gate and no verdict claimed goes against us on all three axes we could measure: at matched fidelity CFG keeps more diversity, the strength knob is not monotone because the rotation wraps, and the base model fits 33% worse before any guidance. We therefore claim less distortion at equal cost on a deterministic target, and no more.

The experiment that would falsify the account. The rule of §7 predicts a *location*, not a direction. Build a task family whose conditional operator has a dialable stretch-to-rotation ratio, $T^*(c) = U(c)^\alpha S(c)^{1-\alpha}$; the closed forms then predict where FiLM, Complex FiLM and a rotation each take over as α sweeps. Registering those crossover points before running would convert a framework fitted on nine observations into a prediction tested out of sample, and would predict our own losses. We have not run it.

A prediction we record before testing it. World models condition on past proprioception and on past actions, usually through the same mechanism. The account says they want different ones. A proprioceptive reading names which features should be active, which is content and lives in the stretch factor, and proprioceptive states do not compose, so closure has nothing to determine; FiLM should win there. Actions compose by construction, so the group structure is real, and on our own rollout suite a composing operator with a consistency loss beat FiLM by 78% at horizon 20. We predict that splitting the two paths beats using one mechanism for both, that making both FiLM costs long-horizon accuracy, and that making both an operator costs single-step fit. It is a smaller intervention than anything we built, it tests the account and not the operator, and we state it here so that it can fail in public.

Reproducibility statement. We release the code, the pre-registrations, and the raw logs. Every suite has a specification we wrote before its decision run, and it is worth saying exactly how much of that a reader can check. We registered with version control and not with a third-party registry, so the record is our repository’s history. For ten of sixteen suites the specification sits in an earlier commit than its results, by one hour to four days. For the other six the specification and its results landed in the same commit, so the history is consistent with the ordering we describe but does not establish it; the composition suite, whose AC-2 criterion this paper leans on, is one of the six. No specification was ever committed after its results. Future work of ours will commit the specification separately and record its hash in the run output, which makes the ordering checkable instead of asserted. A registry maps each suite label here to the module that produced it, the log it wrote, and its verdict, so a replicator reproduces any row with a single command. Every number, table, and figure regenerates from those append-only logs. Unit tests pin the operator invariants (identity at initialization and at $c = 0$, orthogonality, exact composition), the budget counters, and the registry itself. All amendments and one accounting erratum, a FLOP undercount our own audit caught that downgraded a favorable verdict, we disclose with the results.

References

- [1] E. Perez, F. Strub, H. de Vries, V. Dumoulin, A. Courville. FiLM: Visual reasoning with a general conditioning layer. *AAAI*, 2018.
- [2] W. Peebles, S. Xie. Scalable diffusion models with transformers. *ICCV*, 2023.
- [3] D. Ha, A. Dai, Q. Le. Hypernetworks. *ICLR*, 2017.
- [4] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv:2104.09864*, 2021.
- [5] L. Zhang, K. Chen, X. Liu, et al. Group representational position encoding. *ICLR*, 2026.
- [6] E. Hu et al. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.
- [7] Z. Qiu et al. Controlling text-to-image diffusion by orthogonal finetuning. *NeurIPS*, 2023.
- [8] W. Liu et al. Parameter-efficient orthogonal finetuning via butterfly factorization. *ICLR*, 2024.
- [9] S. Yuan, H. Liu, H. Xu. Householder reflection adaptation. *NeurIPS*, 2024.
- [10] M. Gorbunov et al. Group and shuffle: Efficient structured orthogonal parametrization. *NeurIPS*, 2024.
- [11] J. He, C. Zhou, X. Ma, T. Berg-Kirkpatrick, G. Neubig. Towards a unified view of parameter-efficient transfer learning. *ICLR*, 2022.
- [12] M. Montero, C. Ludwig, R. Costa, G. Malhotra, J. Bowers. The role of disentanglement in generalization. *ICLR*, 2021.
- [13] I. Higgins et al. Towards a definition of disentangled representations. *arXiv:1812.02230*, 2018.
- [14] T. Cohen, M. Welling. Group equivariant convolutional networks. *ICML*, 2016.
- [15] R. Quessard, T. Barrett, W. Clements. Learning disentangled representations and group structure of dynamical environments. *NeurIPS*, 2020.
- [16] H. Caselles-Dupré, M. Garcia-Ortiz, D. Filliat. Symmetry-based disentangled representation learning requires interaction with environments. *NeurIPS*, 2019.
- [17] D. Worrall, S. Garbin, D. Turmukhambetov, G. Brostow. Interpretable transformations with encoder-decoder networks. *ICCV*, 2017.
- [18] H. Keurti, A. Neitz, S. Weichwald, B. Schölkopf, et al. Homomorphism autoencoder: Learning group structured representations from observed transitions. *ICML*, 2023.

- [19] N. Hansen, X. Wang, H. Su. Temporal difference learning for model predictive control. *ICML*, 2022.
- [20] E. Chan, M. Monteiro, P. Kellnhofer, J. Wu, G. Wetzstein. pi-GAN: Periodic implicit generative adversarial networks. *CVPR*, 2021.
- [21] Z. Sun, Z. Deng, J. Nie, J. Tang. RotatE: Knowledge graph embedding by relational rotation in complex space. *ICLR*, 2019.
- [22] X. Zhu, C. Xu, D. Tao. Commutative Lie group VAE for disentanglement learning. *ICML*, 2021.
- [23] N. Liu, S. Li, Y. Du, A. Torralba, J. Tenenbaum. Compositional visual generation with composable diffusion models. *ECCV*, 2022.
- [24] A. Bradley, P. Nakkiran. Classifier-free guidance is a predictor-corrector. *arXiv:2408.09000*, 2024.
- [25] M. Fu et al. TeEFusion: Blending text embeddings to distill classifier-free guidance. *ICCV*, 2025.
- [26] T. Nagarajan, K. Grauman. Attributes as operators. *ECCV*, 2018.
- [27] M. Georgopoulos, G. Chrysos, M. Pantic, Y. Panagakis. Multilinear latent conditioning for generating unseen attribute combinations. *ICML*, 2020.